

Workshop: Loops

The purpose of this workshop is to practise construction of programs with loops. All the examples and exercises in this workshop will result in a drawing, so you will have immediate feedback on your work.

All the examples and exercises in this paper are built around the same template:

- A method named `drawShapes()` – the method makes a drawing.
- A class named `Gui` – the class shows the drawing in a window.
- A class named `App` with the `main()` method.

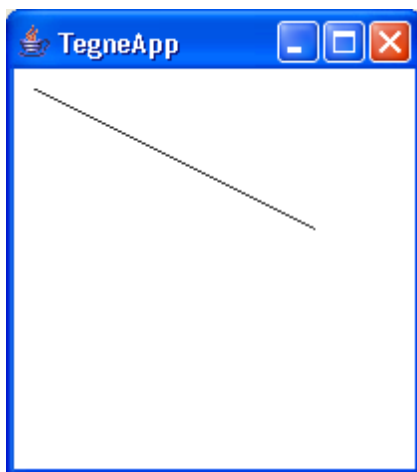
To create a program that makes a suitable drawing, *you will only have to alter the `drawShapes()` method*. The other parts of the `Gui` class must stay unchanged.

Straight lines

Example 1

If you run the `App` program in the handed-out *loopstemplate* package, you will see a window with a single line. To make a drawing, you will have to make a new package (with a suitable name), copy the classes in the template package to the new package, and change the code in the `drawShapes()` method.

Here is an example (suitable package name: `example1`). The following `drawShapes()` method will draw a straight line from (10,10) to (150,80):



```
private void drawShapes(Pane pane) {  
    Line line = new Line(10, 10, 150, 80);  
    pane.getChildren().add(line);  
}
```

In all the exercises you will have to place your code to the `drawShapes()` method (do not change anything else).

The method `drawShapes()` uses an object named *line* of type *Line*. The method can use objects of other types, i.e. *Circle*, *Ellipse*, *Rectangle*, *Arc*, *Polygon*, *Text*.

Example 2

Things really become interesting when you use variables in your code.

The window below shows 2 arrowheads. The code is shown beside the window.



```
private void drawShapes(Pane pane) {  
    // draw an arrowhead at (50,30)  
    int x = 50;  
    int y = 30;  
    Line line1 = new Line(x, y, x + 10, y - 4);  
    Line line2 = new Line(x, y, x + 10, y + 4);  
    pane.getChildren().addAll(line1, line2);  
  
    // draw an arrowhead at (25,140)  
    x = 25;  
    y = 140;  
    Line line3 = new Line(x, y, x + 10, y - 4);  
    Line line4 = new Line(x, y, x + 10, y + 4);  
    pane.getChildren().addAll(line3, line4);  
}
```

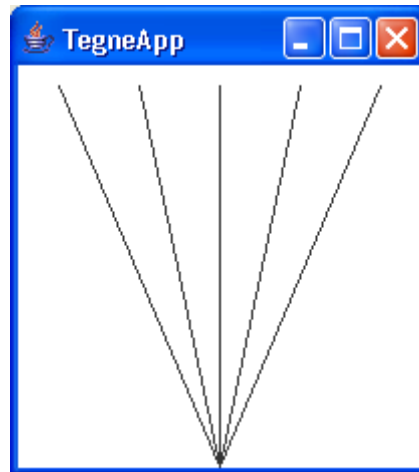
With variables for the positions, it is easy to move the arrowheads around, or add new arrowheads. Try it in the exercise below.

Exercise 1

- Make an application that draws two arrowheads at (100,75) and (100,125).
 - Add some code to make the application show a third arrowhead at (20,50).
 - Change the code, so all the arrowheads have twice the size.
- Use variables to make it easy to change the size (both horizontally and vertically) of all arrowheads (you can assume that all arrowheads have the same size).

Example 3

The use of variables is especially important in connection with iterations (repetitions). We will draw the figure below.



As you can see all the lines in the figure have a shared start point (the lowest point in the figure). This point is at (100,200). The final points of the lines are at (20,10), (60,10), (100,10), (140,10) and (180,10).

To draw the leftmost line, we will use four variables to hold the coordinates of the line's end points:

```
int x1 = 100; // start point: (x1,y1)
int y1 = 200;
int x2 = 20; // end point: (x2,y2)
int y2 = 10;
```

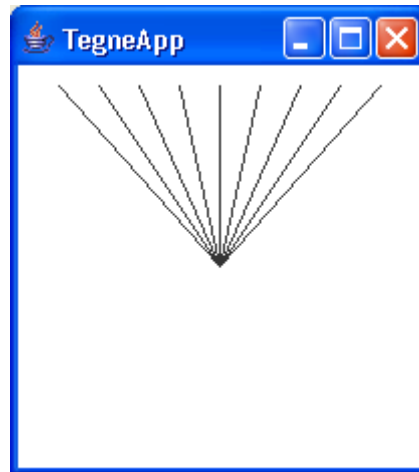
All the other lines have the same value for x1, y1 and y2 as the first line. Only x2 must be changed for the other lines. For each new line, you draw to the right of the first line in the figure, you will have to increase x2 with 40.

To repeat the drawing of a line and repeat the adding of 40 to x2, we will use a while statement:

```
private void drawShapes(Pane pane) {
    int x1 = 100; // start point: (x1,y1)
    int y1 = 200;
    int x2 = 20; // end point: (x2,y2)
    int y2 = 10;
    while (x2 <= 180) {
        Line line = new Line(x1, y1, x2, y2);
        pane.getChildren().add(line);
        x2 += 40;
    }
}
```

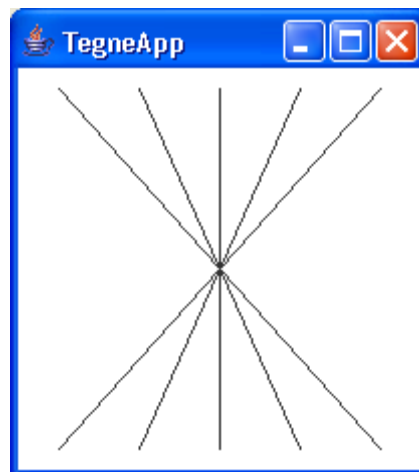
Exercise 2

Create an application like the one above, but this time with 9 lines and with a shared start point at (100,100):



Example 4

We will now draw the following figure:



Start points:

(20,190)
(60,190)
(100,190)
(140,190)
(180,190)

End points:

(180,10)
(140,10)
(100,10)
(60,10)
(20,10)

We can use almost the same method as in example 3.

The value of x1 will be 20, 60, 100, 140 and 180, and the value of x2 will be 180, 140, 100, 60, and 20. y1 will be 190, and y2 will be 10.

The code below draws the figure:

```
private void drawShapes(GraphicsContext gc)
{
    int x1 = 20; // start point: (x1,y1)
    int y1 = 190;
    int x2 = 180; // end point: (x2,y2)
    int y2 = 10;
    while (x1 <= 180) {
        Line line = new Line(x1, y1, x2, y2);
        pane.getChildren().add(line);
        x1 += 40;
        x2 -= 40;
    }
}
```

The code in examples 3 and 4 have the same structure:

```
<declare and initialize the variables x1, y1, x2, y2>
while (<condition involving one or more of the variables>) {
    Line line = new Line(x1,y1,x2,y2);
    pane.getChildren().add(line);
    <update one or more of the variables>
}
```

Here is the way to think about the code:

- 1) Choose the line you want to draw as the first line.
Fill in **<declare and initialize the variables x1, y1, x2, y2>** with the coordinates of this line.
- 2) Find the variable(s) that need new values for the next line.
Fill in **<update one or more of the variables>**.
- 3) Find the condition on the variable(s) that can stop the loop, when it is no longer true.
Fill in **<condition involving one or more of the variables>**.

Let's look at example 4 and follow this prescription:

- 1) The line slanting upwards and to the right is drawn first.
<declare and initialize the variables x1, y1, x2, y2> will become:

```
int x1 = 20; //start point: (x1,y1)
int y1 = 200;
int x2 = 180; //end point: (x2,y2)
int y2 = 10;
```

- 2) For the next line x1 have to be increased by 40 and x2 has to be decreased by 40.

<update one or more of the variables> will become:

```
x1 += 40;
x2 -= 40;
```

- 3) New line swill be drawn as long as $x1 \leq 180$.
<condition involving one or more of the variables> will become:

```
x1 <= 180
```

Exercise 3

Create 3 applications that draw the following figures.

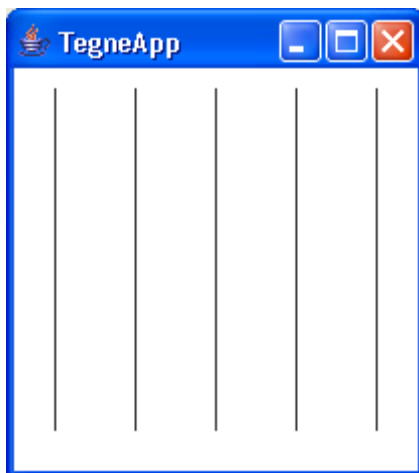


Figure 1



Figure 2

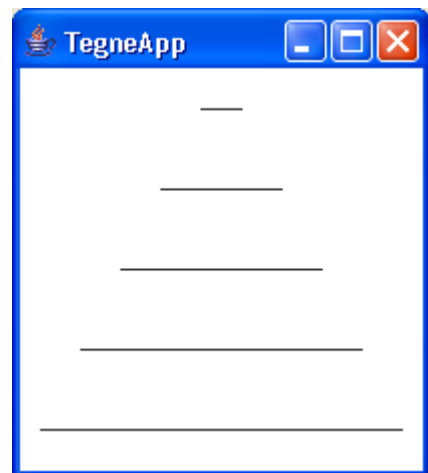


Figure 3

Hint:

In figure 1, x1 and x2 will have the same value, so you need only one x-variable.

Infinite loop

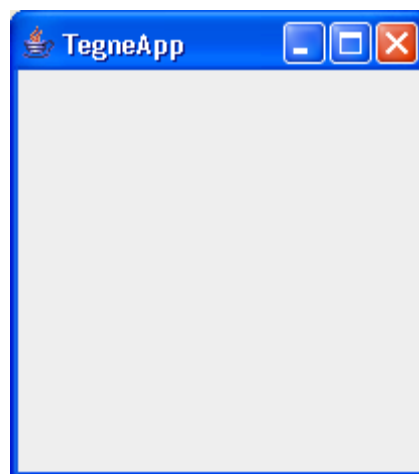
A possible error involving loops is an infinite loop. An infinite loop will never end.

If you change example 3 to

```
public void drawShapes(Pane pane) {  
    int x1 = 100;  
    int y1 = 200;  
    int x2 = 20;  
    int y2 = 10;  
    * while (x2 > 0) {  
        Line line = new Line(x1, y1, x2, y2);  
        pane.getChildren().add(line);  
        x2 += 40;  
    }  
}
```

then the code has an infinite loop. The condition $x2 > 0$ will always be true, because $x2$ starts with the value 20 and is always increased (by 40).

When you run this application, you will not see any figure in the window (or you will not see the window):



The figure will not be shown, because the figure is transferred to the window when the loop is done. And the loop is never done!

An added complication is that you cannot close the window by clicking at the close icon in the upper, right corner. Instead, you will have to end the application by clicking on the red square in IntelliJ:



Circles and Ellipses

Example 5

Now we will draw circles and ellipses. To do that you will have to use objects of the Circle and Ellipse type.

As an example, the code

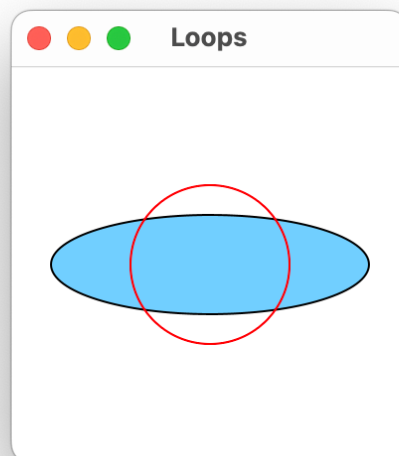
```
Ellipse ellipse = new Ellipse(100, 100, 80, 25);  
ellipse.setStrokeColor(Color.BLACK);  
ellipse.setFill(Color.LIGHTSKYBLUE);
```

will draw an ellipse with center at (100,100) and x-radius 80 and y-radius 25. The ellipse's perimeter is colored black, and the interior of the ellipse is colored light-sky-blue.

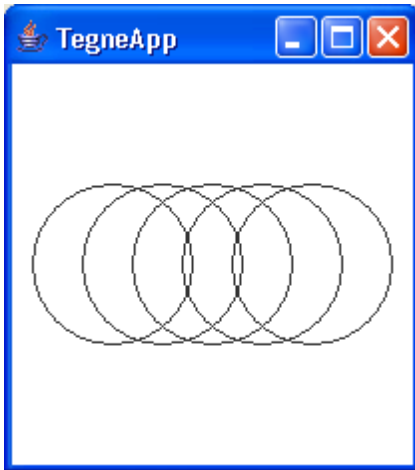
The code

```
Circle circle = new Circle(100, 100, 40);  
circle.setStrokeColor(Color.RED);  
circle.setFill(null);
```

will draw a circle with center at (100,100) and radius 40. The circle's perimeter is colored red, and the interior is not colored at all (*null* means no color). The ellipse behind the circle is visible, because the interior of the circle has no color.



Example 6



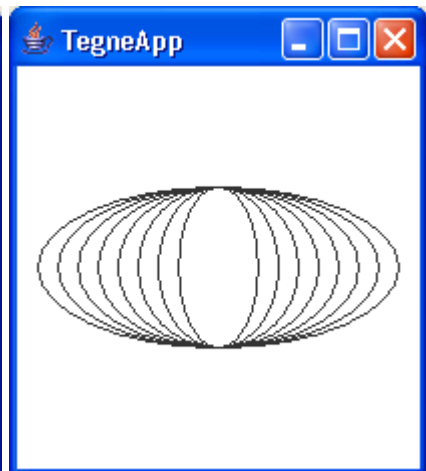
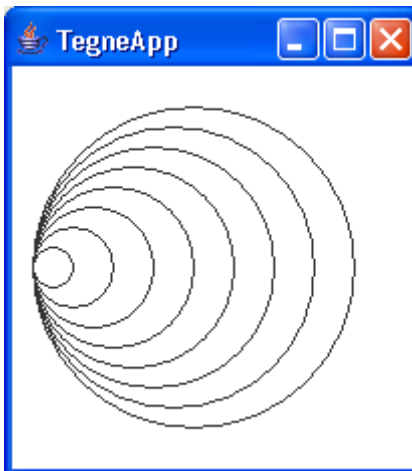
With the following code you can draw a family of circles:

```
private void drawShapes(Pane pane) {  
    int x = 50; // center: (x,y)  
    int y = 100;  
    int r = 40; // radius: r  
    while (x <= 150) {  
        Circle circle = new Circle(x, y, r);  
        circle.setStroke(Color.black);  
        circle.setFill(null);  
        pane.getChildren().add(circle);  
        x += 25;  
    }  
}
```

Exercise 4

Create the figures shown below.

Another important hint: *Compute the numbers (center and radius) of the circles, before you make your code. A drawing on paper with numbers might be helpful.*



Exercise 5

Text can be drawn with an object of type *Text*.

As an example the code below will write the text "Datamatiker" placed with lower, left corner at (20,50):

```
String str = "Datamatiker";  
Text text = new Text(str, 20, 50);
```

Create an application that draws this figure:



Hint:

Create a variable named *str* of type *String* to hold the word "Datamatiker".

To get a substring of a string you can use the method *substring()* in the *String* class..

If the value of the variable *str* is "Datamatiker" then

`str.substring(0,1)` is "D"

`str.substring(0,2)` is "Da"

`str.substring(0,3)` is "Dat" etc.

Code a while-statement that repeats the following:

- Select a substring of *str*.
- Write the substring at (x,y).
- Change the value of *y* and the substring for the next iteration.

You will need a variable to count the number of characters in the substring.

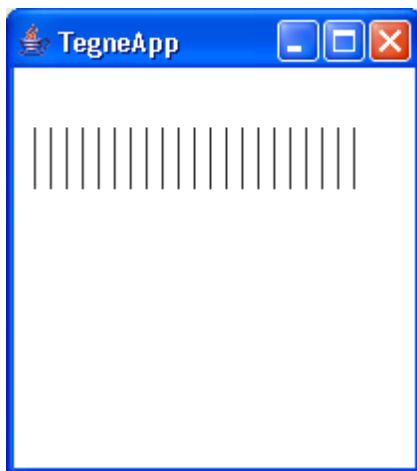
If you find it difficult to make this exercise, you can go another way: Start by creating an application that writes the word "Datamatiker" 11 times (each word on a new line). Then change the code, so the substrings are written instead of the whole word.

for-statements

Sometimes your code will be more readable if you use a for-statement instead of a while-statement. A for-statement is the natural choice, when you know that code lines must be repeated a certain number of times. That is, when you know the boundaries of the index variable in the for-statement (e.g. the value of the index variable is in $0 \dots 10$, or in $0 \dots \text{length}-1$, or in $\text{lower} \dots \text{upper}$).

Example 7

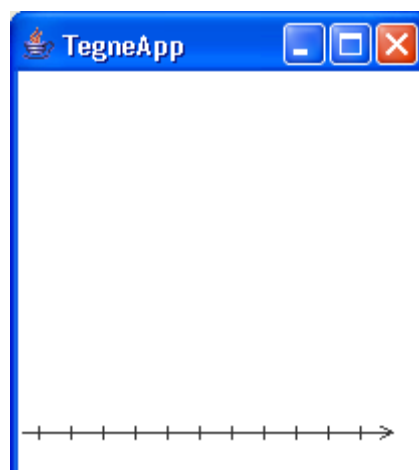
We want to draw 20 vertical lines with a distance of 8. The first line has to be at $x = 10$. So, the x-values of the lines will be 10, 18, 26, 34, and so forth. These x-values fit in with the formula $10 + i * 8$, where i is the index variable: $i = 0, 1, 2, \dots, 19$. A way to make the drawing is to use a for-statement:



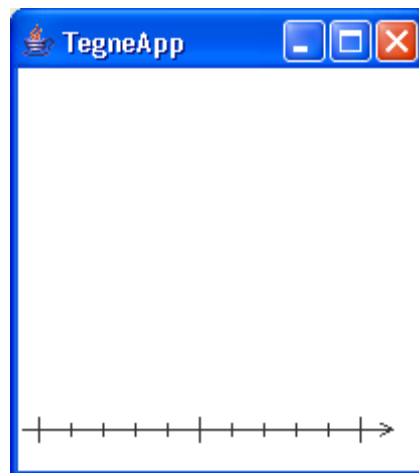
```
private void drawShapes(Pane pane) {  
    int y1 = 30; // start point: (x,y1)  
    int y2 = 60; // end point: (x,y2)  
    for (int i = 0; i < 20; i++) {  
        int x = 10 + i * 8;  
        Line line = new Line(x, y1, x, y2);  
        pane.getChildren().add(line);  
    }  
}
```

Exercise 6

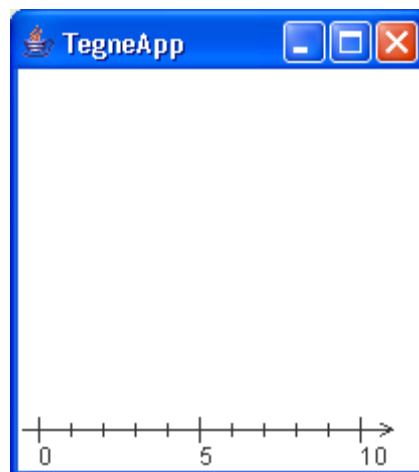
Create an application that draws an x-axis as shown below. The application must use a for-statement (the x-axis has 11 ticks).



Add some code to make each fifth vertical stroke twice as long as the others:



Add more code to write numbers at the long strokes:

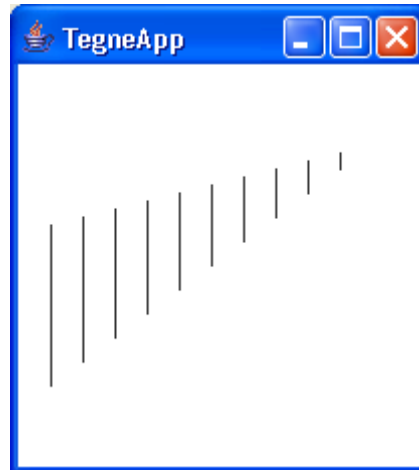


Exercise 7

Repeat exercise 5, but this time you must use a for-statement instead of a while-statement.

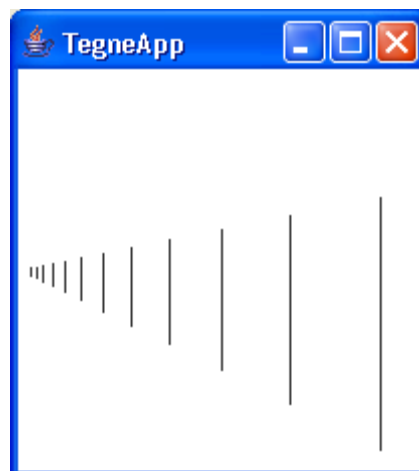
Exercise 8

Create an application that draws 10 vertical lines as shown in the figure below. The first line's height is 80 and the height of the next lines is decreased by 8. The lower endpoint of the lines is at (16,160), (32,148), (48,136), etc. Use a for-statement.

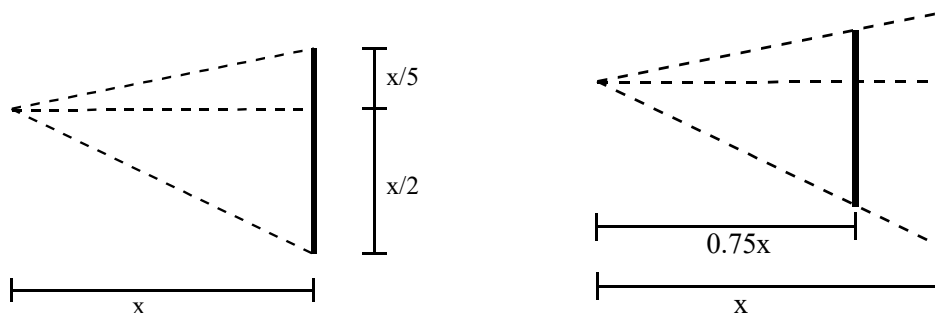


Exercise 9

Create an application that draws the following picture (use a while-statement):



Hint: The figures below illustrate how a line is drawn.

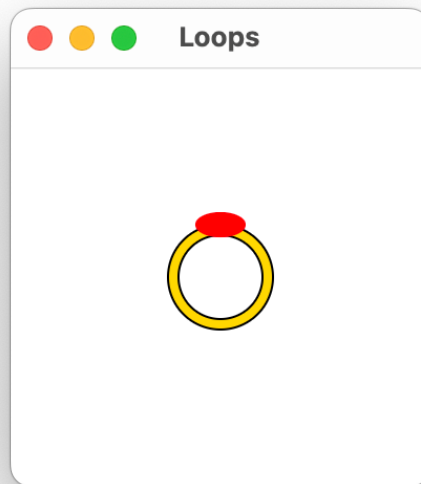


Draw the rightmost line first (e.g. at $x=180$). Decrease the x -value of each new line with 25 %. Stop when x is below 5.

Methods

Exercise 10

We can draw a ring by drawing 2 circles and an ellipse (red) as shown here.



If we want to draw several rings, the code will be complicated. We can get a much clearer code if we use methods.

Make a method that draws a ring with center at (x,y):

```
private void drawRing(Pane pane, int x, int y)
```

The method drawRing() has three parameters: pane, x and y, where (x,y) is the center of the ring.

To draw four rings at (50,50), (150,50), (50,150) and (150,150), we will change the code inside drawShapes() to:

```
private void drawShapes(Pane pane) {  
    drawRing(pane, 50, 50);  
    drawRing(pane, 150, 50);  
    drawRing(pane, 50, 150);  
    drawRing(pane, 150, 150);  
}
```

Exercise 11

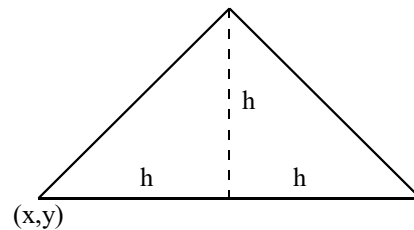
Extend the application from example 6 with a method that draws a cross with center at (x,y) and size w (size is the length of the lines in the cross).

+

Use the method to draw crosses at the center of the circles.

Exercise 12 *

Create an application that draws several triangles. You must code a method that draws a triangle. The triangle must have (x,y) and h as parameters (look at the figure).



Use your triangle-method to draw the following figures. The size of the small triangles is $1/3$ of the big triangle. ($h = 81$ is used in the figure).

Hint: Make a method `drawInnerTriangles()` that draws 3 triangles inside a triangle given by (x,y) and h .

